litefont

A small text rendering library for OpenGL, Direct3D and the CPU

Shane Transue

1 Overview

litefont is a small text rendering library: it turns OpenType fonts into crisp text. It reads the font file itself, renders hinted glyph images with exact antialiasing, lays out paragraphs and labels, and draws them through whichever graphics API a program uses (OpenGL 1.1, an OpenGL 3.3 core profile, Direct3D 11) or into an image on the CPU. It is written in C++20; its core depends only on the standard library, and each graphics API sits behind one device class, so no extension loader or other library is needed.

- **Fonts:** TrueType and CFF (`.otf`) outlines, collections (`.ttc`), kerning from GPOS or the kern table, names and traits; damaged files fail cleanly.
- **Families:** regular, bold, italic and bold italic faces, simulated when missing; fallback families for missing code points; a catalog that finds families by name in a folder.
- **Glyphs:** exact-area coverage with vertical hinting; outlines, sharp and blurred shadows, all cached in a texture atlas; gradients with a color in each corner.
- **Layout:** wrapping, alignment, tabs, letter spacing, line limits with an ellipsis, superscripts and subscripts, underline and strikethrough, per-range colors and styles, anchors and quarter-turn rotation.
- **Editing queries:** caret boxes, selection rectangles, hit tests and caret movement.

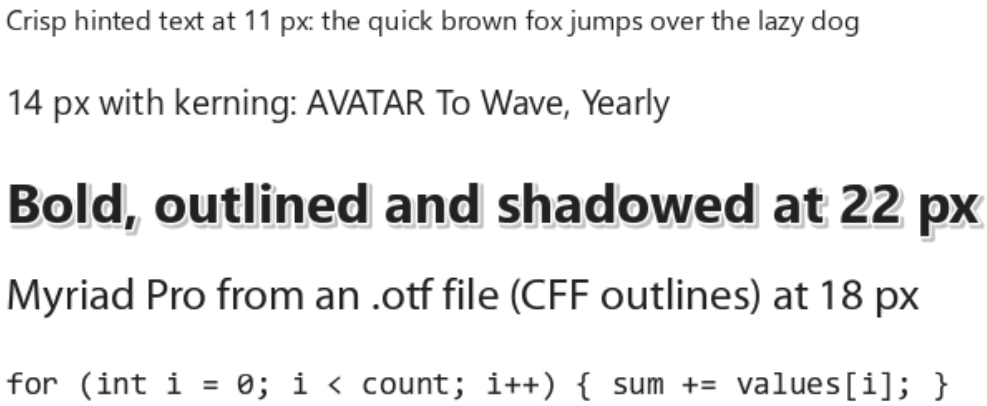


Figure 1: Text drawn by litefont on the CPU at 150 percent scale: hinted small text, kerning, a bold outlined and shadowed label, a CFF font and monospaced code.

2 Quick start

Every program does the same three things: it declares a font, the device for its graphics API and a painter that draws through the device; it loads the font once; and it calls `drawText` wherever it already draws.

OpenGL

`OpenGL11Device` draws in any compatibility context, next to `glBegin` and `glEnd`; `OpenGL33Device` draws in a 3.3 or newer context, core profiles included. Either can be declared before the context exists: it finds its OpenGL functions and makes its textures at the first draw, so no loader is needed.

```
#include <litefont/litefont.h>

litefont::Font font;                        // (1) declare
litefont::OpenGL11Device device;            // or OpenGL33Device
litefont::TextPainter painter(device);

void init() {
    font.load("NotoSans-Regular.ttf");      // (2) load once
}

void display() {
    glBegin(GL_TRIANGLES);                  // the program's own drawing
    // ...
    glEnd();
    painter.drawText(font, 16.0f, "Hello world", 20.0f, 40.0f);   // (3) draw
}
```

Direct3D 11

`D3D11Device` draws with the program's own Direct3D device and context, into whatever render target is bound and in its first viewport. It can be declared before Direct3D exists and attached once the program has made its device; it makes its shaders, states and textures at the first draw.

```
#include <litefont/litefont.h>

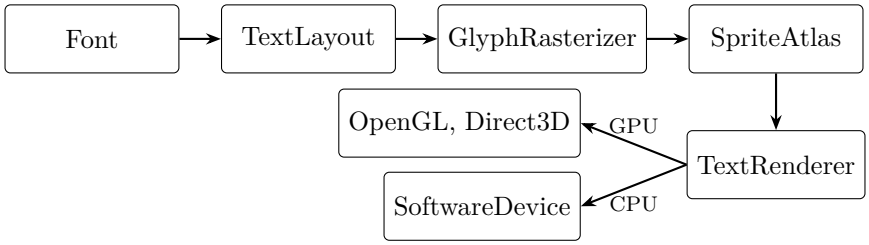
litefont::Font font;                        // (1) declare
litefont::D3D11Device device;
litefont::TextPainter painter(device);

void init(ID3D11Device* d3d, ID3D11DeviceContext* context) {
    device.attach(d3d, context);            // once Direct3D exists
    font.load("NotoSans-Regular.ttf");      // (2) load once
}

void render() {
    context->OMSetRenderTargets(1, &target, nullptr);
    context->RSSetViewports(1, &viewport);
    // ... the program's own drawing ...
    painter.drawText(font, 16.0f, "Hello world", 20.0f, 40.0f);   // (3) draw
    swapChain->Present(1, 0);
}
```

Positions are pixels from the top-left corner of the viewport. `measureText` gives the box a string takes, for centering or right alignment, and `setScale` sets the device pixels per pixel on high-density screens. The same painter also draws on the CPU: with a `SoftwareDevice`, the program calls `beginFrame` with a size and a clear color before drawing, and `readPixels` afterwards to take the image. The smallest complete programs, a window that says "Hello world", are `examples/hello_glut` and `examples/hello_d3d11`.

3 How it works



A **Font** owns its file's bytes and answers questions about them: metrics, the glyph for a code point, advances, kerning and outlines. **TextLayout** places the glyphs of a **Text** (its content, family, size, colors and layout settings) on whole-pixel baselines. The **GlyphRasterizer** renders each glyph at its size, hinted so baselines and x-heights land on whole pixels, and the **SpriteAtlas** keeps the images in texture pages. The **TextRenderer** turns placed glyphs into quads and hands them to a **GraphicsDevice**, the one interface every graphics API implements: **OpenGL11Device**, **OpenGL33Device** and **D3D11Device** on the GPU, **SoftwareDevice** on the CPU. All of them use the same blending and give the same pixels to within one level.

A Text is laid out once and kept. Drawing it again, or moving it, costs no layout and no glyph rendering; only a change to how it looks starts over.

4 Devices and drawing

`TextPainter` is the immediate front end, over any device. Each `drawText` call draws at once, in call order with the program's own drawing. Behind it, every distinct string with its font, size, style and color keeps its layout and glyphs, so a string drawn every frame, even at a new position, costs a lookup and moving its quads. The least recently drawn strings are dropped once the cache holds more than 1024. `draw(text, x, y)` does the same for a Text with all its own settings.

The program names its device once, where it declares it:

- `OpenGL11Device` uses only OpenGL 1.1 fixed function, so it runs in any compatibility context, next to `glBegin` and `glEnd`. Glyph pages are alpha textures, modulated by the vertex color. It saves and restores the program's state with `glPushAttrib`, `glPushClientAttrib` and the matrix stacks.
- `OpenGL33Device` runs in 3.3 or newer contexts, core profiles included. It draws with one small shader from a vertex array and buffers, with one-channel glyph pages read as alpha. Core profiles have no attribute stack, so it reads each value it changes and sets it back afterwards.
- `D3D11Device` draws with Direct3D 11 into whatever render target is bound. The program hands it its `ID3D11Device` and context; its shaders are compiled ahead of time and built in, and it saves and restores every piece of pipeline state it sets.
- `SoftwareDevice` draws into an image in memory, the way a GPU does: the reference the others are checked against, and the device for tests and command-line rendering.

For each draw a GPU device sets what text needs (a projection, blending, the atlas texture, the scissor for clips) and switches off depth test and face culling. `setRestoreState(false)` skips saving and restoring for programs that manage their own state.

The devices stay out of each other's way. The OpenGL devices include no OpenGL header and need no loader: each finds its functions at the first draw through a table of them (`OpenGL11Functions`, `OpenGL33Functions`), and `OpenGLFunctions.cpp` is the only file that knows the operating system. `D3D11Device` calls Direct3D only through the objects the program gives it, so it links no Direct3D library, and its header declares the Direct3D interfaces without including a Windows header. Textures, buffers and shaders are made at the first draw; `release()` deletes them, and `getError()` says why a device drew nothing.

Text in the program's own transform

By default text sits on the pixel grid of the viewport. To draw it in the program's own coordinates instead, such as a zoomable 2D view, a turning label or a point in a 3D scene, the program tells the device: `setMatrix(m)` takes a column-major matrix (projection times model-view, as a shader would) on any device, the CPU included, and on `OpenGL11Device`, `useMatrixStack()` draws through whatever the program set with `glOrtho`, `glRotatef` and the rest. `useViewport()` goes back to pixels.

```
glMatrixMode(GL_MODELVIEW);
glRotatef(degrees, 0.0f, 0.0f, 1.0f);
device.useMatrixStack();
painter.draw(label);
device.useViewport();
```

Positions and sizes are then in the program's units, and text grows downward from its position; y is up by default, and `setYUp(false)` serves a y-down space. Each string is rendered at about the size it covers on screen, in steps of a sixth of an octave, and the matrix carries it the rest of the way with linear filtering, so text stays sharp at any zoom, and moving, turning or zooming it costs no new layout until its size crosses a step.

5 Texts and labels

A `Text` holds everything about how a piece of text looks. Changing a setting marks it for a new layout, except the offset and clip, which only move or cut what is drawn.

Setting	Effect
content, family, size	UTF-8 text, the faces it uses, its size in pixels
style, style runs	a face per range of bytes, superscript or subscript, underline, strikethrough
color, color runs	a color per range of bytes, such as syntax colors
gradient	a color for each corner of the box, blended across the glyphs
outline, shadow	a border around each glyph; an offset shadow, optionally blurred
background	a box behind the text, grown by padding
max width, align, line spacing, tab width	paragraph layout
letter spacing, max lines, ellipsis	spacing between glyphs; cutting long text with an ellipsis
anchor, rotation	the point of the box that lands on the position; a quarter turn
offset, clip	where it is drawn and what it may cover, with no new layout

`TextCursor` answers the geometry an editor needs from a layout: the caret box, selection rectangles, the position a point hits, and where the caret moves by character, word or line.

6 Fonts

A `Font` loads a `.ttf`, `.otf` or one face of a `.ttc` file, from a path or from bytes in memory. A `FontFamily` groups up to four faces and simulates bold and italic where a face is missing; fallback families supply code points the family lacks, and the replacement character marks those nobody has. A `FontCatalog` reads only the names and traits of the fonts in a folder, from a few small reads per file, and loads a family by name. litefont never looks for system fonts by itself: it reads the files and folders a program names.

7 Building

litefont builds with CMake 3.20 or newer and any C++20 compiler; with CMake 3.21 or newer, its presets do the setup.

```
cmake --preset default
cmake --build --preset release
```

That builds the library, its tests and the examples in `build/` (programs in `build/bin`, libraries in `build/lib`) and assembles `build/bin_release`: `examples/` holds the example programs, ready to start with a double-click, with their DLLs and the open-source Noto fonts they ship with, and `sdk/` holds `include/` and `lib/` with the library and its CMake package. The `vcpkg` preset takes packages such as freeglut from `VCPKG_ROOT`, and the `asan` preset builds with AddressSanitizer in `build-asan/`.

`LITEFONT_OPENGL` and `LITEFONT_D3D11` turn the GPU devices off; with both off, litefont depends on the C++ standard library alone. A project adds litefont with `add_subdirectory`, or uses the SDK (or an install) with `find_package(litefont)`; either way it links `litefont::litefont`.